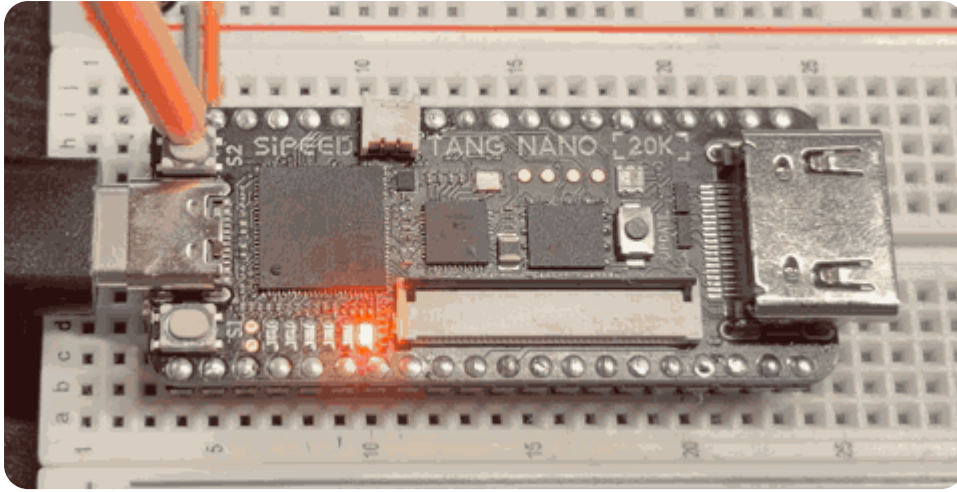


TTA-8 せつめい書 — 8bit の小さい CPU を 作って 動かそう



この せつめい書では、**命令が 2つ (READ と WRITE) しかない 8bit CPU「TTA-8」** を、FPGA ボード Tang Nano 20K の 中 に 作 っ て 動 か し ま す。小 学 校 高 学 年 く ら い か ら 読 め る よ う に 書 い て あ り ま す。

もくじ

章	内容	こんな人に
1. CPU って なんだろう？	CPU・プログラム・2進数	はじめての人
2. TTA-8 の しくみ	READ と WRITE だけで 計算できる ひみつ	CPU の 中を 知りたい人
3. じゅんびしよう	ボード・ケーブル・道具の 用意	ボードを 買った人
4. 動かしてみよう	LED フラッシャーを ボードに 書き こむ	ボードを 光らせたい人
5. TTA-8 ラボで 中を のぞこう	ブラウザの デバッガの 使いかた	1命令ずつ 見たい人
6. TinyBASIC改 で プログラムを 書こう	BASIC の 文法と サンプル	プログラムを 作りたい 人
7. アセンブリで 書いてみよう	READ / WRITE で 直接 書く	CPU の 気持ちに なりたい 人
8. CPU の 設計図を 読もう	Verilog の文法・クロック・値の保 存	もっと 深く 知りたい人
9. AI に こうやって たのもう	こまったときの AI への 聞きかた	だれでも
10. つぎの ぼうけんへ	ブレッドボードで 回路を ふやそう	ぜんぶ 読んだ人
発展1	わり算・繰り返し・NAND	計算を組み立てたい人
発展2	16bit化・外部メモリ・入出力・ペ ージ	改造の仕組みを考えたい 人
ふろく	こまったとき・用語集・大人の方 へ	保護者・先生

読みかたのヒント ボードがまだなくても、1・2・5・6・7章はブラウザだけでためせます。

このプロジェクトで 使うもの

- Sipeed Tang Nano 20K (FPGA ボード)
- USB ケーブル (データ通信ができるもの。充電専用は×)
- パソコン (Windows か Mac)
- あれば ブレッドボード (ボードを さして 使うと 安定します)

1. CPU って なんだろう？

CPU は「命令を 1つずつ こなす 係」

ゲーム機、スマホ、パソコン、電子レンジ……。その中にはCPU（シーピーユー）という部品が入っています。

CPU がやることは、じつは とても 単純です。

- 1. つぎの 命令を 読む
- 2. その 命令を やる
- 3. 1 に もどる

これを 1秒間に 何十億回も くりかえしているの、ゲームが 動いたり、動画が 見られたり するので

プログラム とは、CPU に やってほしい 命令を 順番に ならべた ものです。料理の レシピ みたいな ものですね。

コンピューターの中は ぜんぶ「数」

CPU は 文字も 絵も 音も、ぜんぶ **数** として あつかいます。しかも その 数は **0 と 1 だけ** で 書かれています。これを **2進数（にしんすう）** と いいます。

LED を 6こ ならべて、光っている ところを 1、消えている ところを 0 と すると……

LED	2進数	ふつうの 数
____○	000001	1
____○_	000010	2
____○○	000011	3
○_○_○_	101010	42
○○○○○○	111111	63

右はしが 1、そのとなりが 2、つぎが 4、8、16、32 …と、左へ 行くたびに **2倍** になります。

101010 は $32 + 8 + 2 = 42$ です。

8bit って？

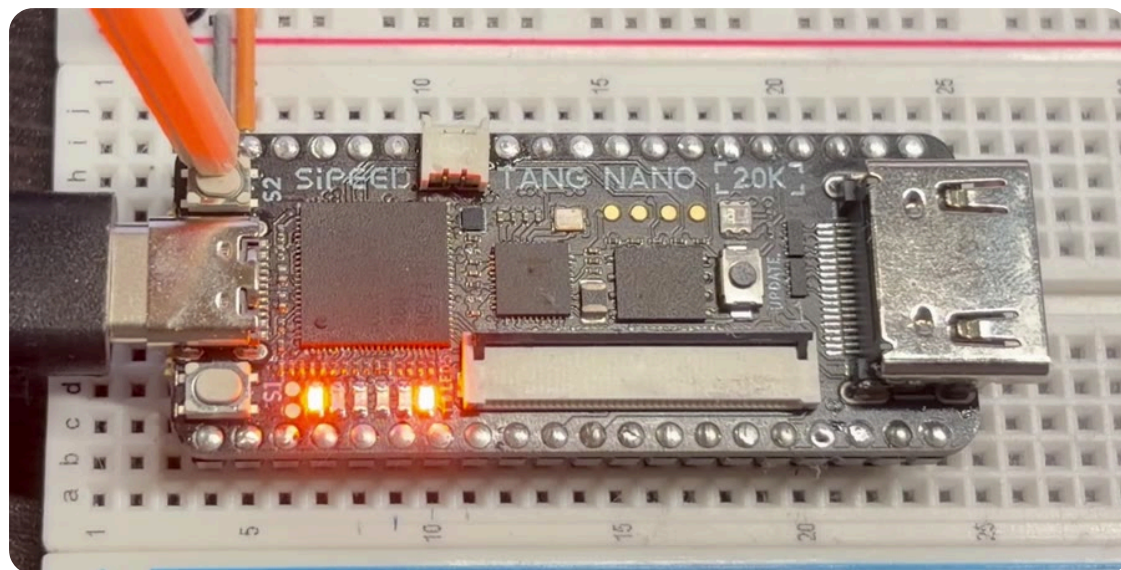
0 か 1 が入る 1マスを **ビット (bit)** と いいます。 **8bit** は 8マス なので、 **00000000** (0) から **11111111** (255) まで、 **256 とおりの 数** を あらわせます。

この せつめい書で 作る CPU「**TTA-8**」は、8bit の 数を あつかう CPU です。 255 に 1を たすと、あふれて 0 に もどります (車の メーターが 一周するのと同じ)。

FPGA は「回路を 書きかえられる チップ」

ふつうの CPU は、工場 で 作られた あとは 中身 を 変えられ ません。でも **FPGA (エフピージーイー)** という チップは、**設計図を 書きこむと、その とおりの 回路に 変身** します。

Tang Nano 20K に のっている FPGA に、TTA-8 の 設計図 (Verilog という ことばで 書いたもの) を 書きこむと、FPGA の 中に **ほんものの CPU** が できあがります。



TTA-8 は **1秒に 1000命令** の ゆっくりした CPU です。 ゆっくりなので、LED の 動きを 目で 追いかけることができます。

2. TTA-8 のしくみ

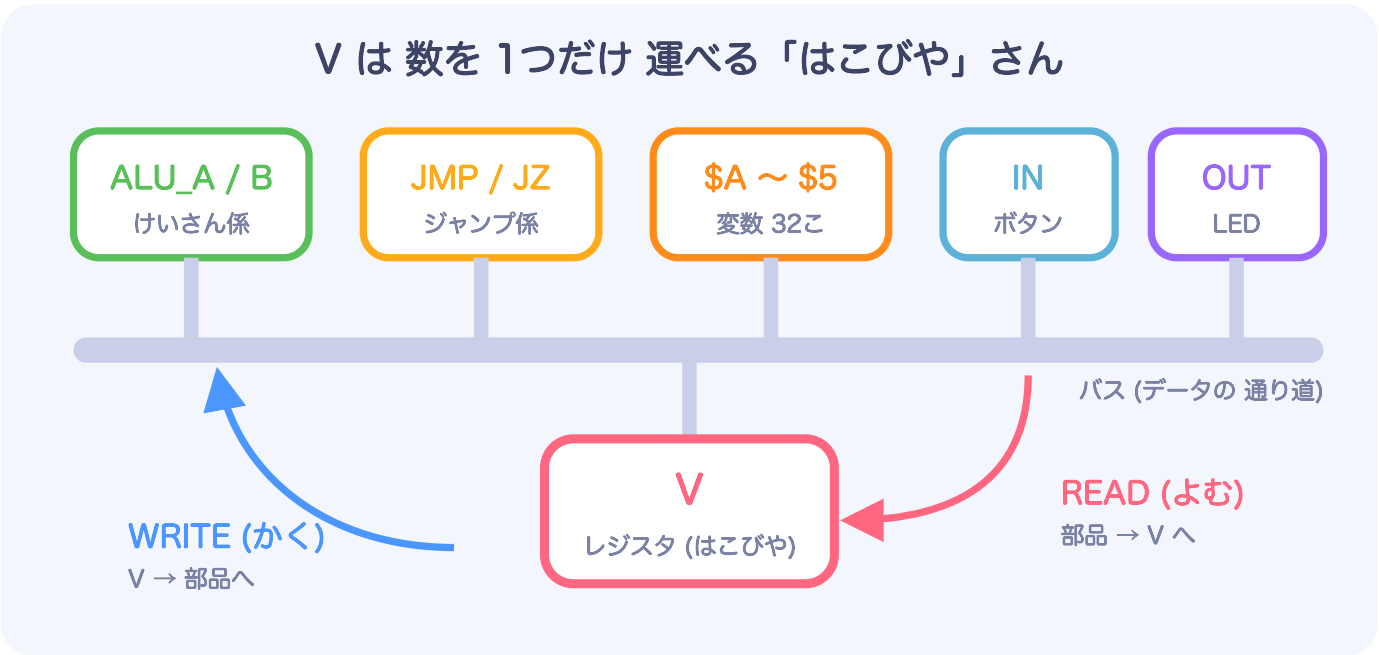
命令は 2 つだけ！

ふつうの CPU には「たし算」「ジャンプ」「比べる」など、何十～何百 種類もの 命令があります。

TTA-8 の 命令は、なんと **2 つだけ** です。

命令	読みかた	すること
READ	よむ	部品から 数を 1 つ 受け取って、V に 入れる
WRITE	かく	V に 入っている 数を、部品に わたす

V (ブイ) は、数を 1 つだけ 持てる「はこびや」さんです (レジスタ といいます)。



「え、それだけで 計算できるの？」と思いますよね。ひみつは **部品の 方が 仕事を する** ことです。けいさん係 (ALU) に 数を わたすと、答えが すぐに 用意されます。V は それを 取りに 行くだけ。この ような CPU を **TTA (Transport Triggered Architecture = 運ぶと 動く しくみ)** と います。

命令の形

TTA-8 の 命令は 1バイト (8bit)



■ 0 = READ (よむ) : アドレスの 値を V へ ■ どの 部品と やりとりするか
1 = WRITE (かく) : V の 値を アドレスへ ALU_A, JMP, OUT, \$A … など

0 1111110 = 0x7E → READ IN (ボタンを よむ)
1 1111111 = 0xFF → WRITE OUT (LED に かく)

命令の指定は 8bit (1バイト) です。READ IMM だけは、次の1バイトに数を置くので、合計2バイトを使います。いちばん 左の 1bit で READ か WRITE かを えらび、のこりの 7bit で「どの 部品と やりとりするか」(アドレス)を えらびます。

部品とアドレスの一覧（アドレスマップ）

アドレス	名前	READ（よむ）と…	WRITE（かく）と…
0x00	ALU_A	A の数	A に 数 を 入れる
0x01	ALU_B	B の数	B に 数 を 入れる
0x02	ALU_ADD	A + B の 答え	—
0x03	ALU_NAND	A NAND B の 答え	—
0x04	ALU_SUB	A − B の 答え	—
0x10	JMP	—	その 番地へ ジャンプ
0x11	JZ	—	A が 0 の ときだけ ジャンプ
0x12	IMM	命令の つぎに 書いた 数	—
0x20～ 0x3F	\$A～\$Z、\$0～ \$5	変数の 数	変数に 数 を 入れる
0x7E	IN	ボタン（S1 = 1、S2 = 2、両方 = 3）	—
0x7F	OUT	—	LED に 出す（1 の ビットが 光 る）

0x が ついた 数は **16進数** です。0x10 は 16、0x7F は 127 の ことです。

ALU（エーエルユー） は Arithmetic Logic Unit（算術論理演算装置）の ことで、CPU の 中で 計算を
する 係です。

3 + 5 を 計算してみよう

```
READ  IMM, 3      ; V に 3 を 入れる
WRITE ALU_A       ; A = 3
READ  IMM, 5      ; V に 5 を 入れる
WRITE ALU_B       ; B = 5
READ  ALU_ADD     ; V = A + B = 8
WRITE OUT         ; LED に 8 を 出す → 001000
```

V が 数 を 運 っ て い る だ け な の に 、 ち ゃ ん と 足 し 算 が で き ま し た 。 LED に は 8 (2 進 数 で 001000) が 光 り ま す 。

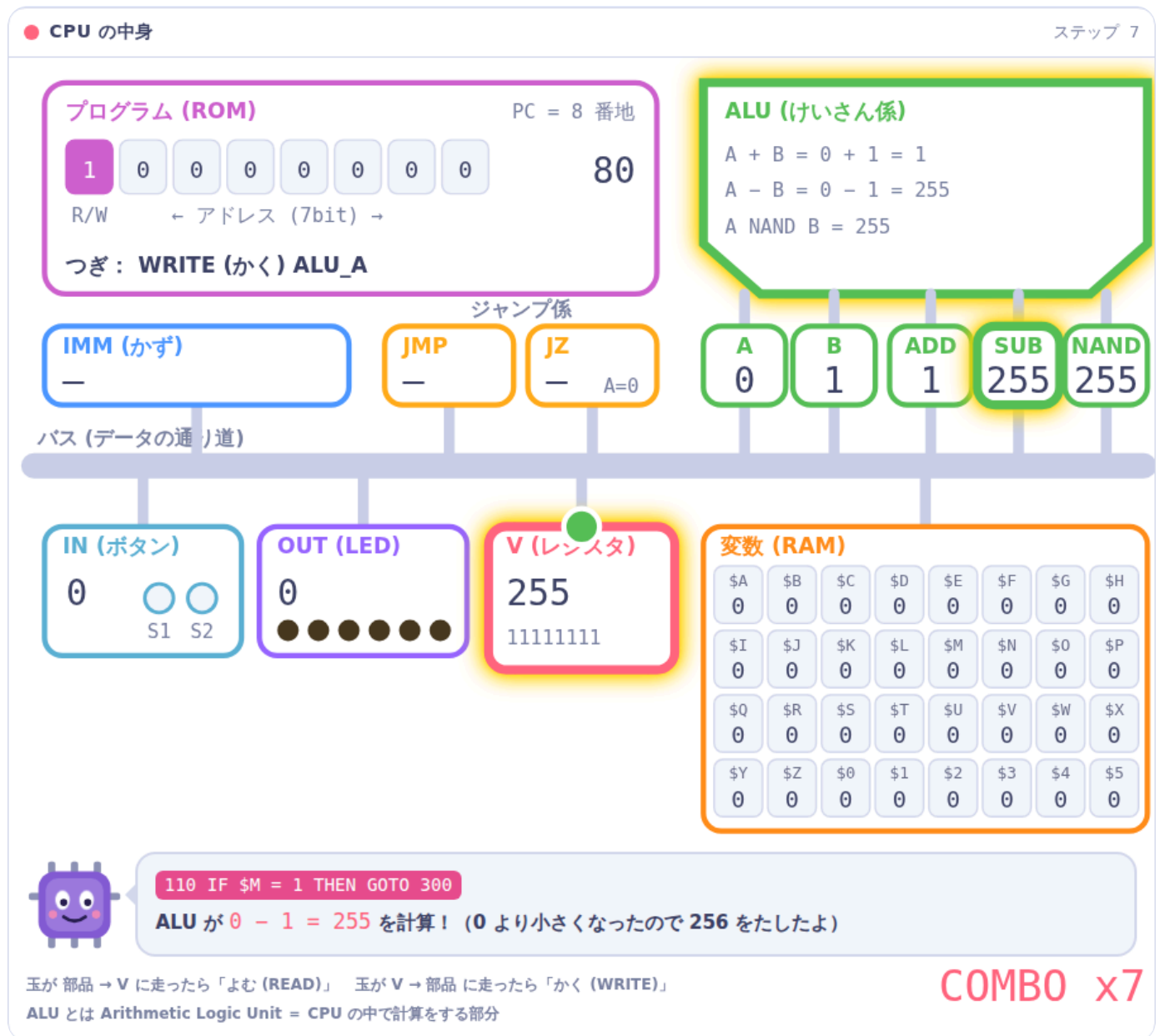
ジャンプも「書くだけ」

CPU は PC (プ ロ グ ラ ム カ ウ ン タ ー) と い う 番 号 で 「 実 行 す る 命 令 の バ イ ト 番 地 」 を お ぼ え て い ま す 。 ふ つ う は 1 つ ず つ 進 み ま す が 、

- JMP に 書 く と 、 PC が そ の 数 に な り ま す (= ジ ャ ン プ)
- JZ に 書 く と 、 ALU_A が 0 の と き だ け ジ ャ ン プ し ま す

JZ が あ る の で 「 0 に な る ま で く り か え す 」 「 ボ タ ン が 押 さ れ て い た ら こ っ ち へ 」 と い っ た 判 断 が で き ま す 。 発 展 1 で は 、 こ の 仕 組 み で わ り 算 や 論 理 演 算 を 組 み 立 て ま す 。

CPU の中身 まとめ



- PC : 実行する命令の バイト番地 (8bit)
- V : 数を 運ぶ はこびや (8bit)
- プログラム (ROM) : 命令と定数を合わせて 256バイトまで 入る
- 変数 (RAM) : \$A~\$Z と \$0~\$5 の 32こ (起動・リセット時はすべて0)
- ALU : A と B から たし算・ひき算・NAND を 計算
- IN / OUT : ボタンと LED

これで ぜんぶです。CPU の 設計図は、コメントと 空行を のぞくと **たった 53行** です。

3. じゅんびしよう

用意するもの

もの	ポイント
Sipeed Tang Nano 20K	FPGA ボード。通販で 買えます
USB ケーブル (USB-C)	データ通信ができる もの 。充電専用だと ボードが 見えません
パソコン	Windows 10 / 11 か Mac
ブレッドボード (あれば)	ボードを さしておくと 安定します。10章 の ぼうけんにも 使います

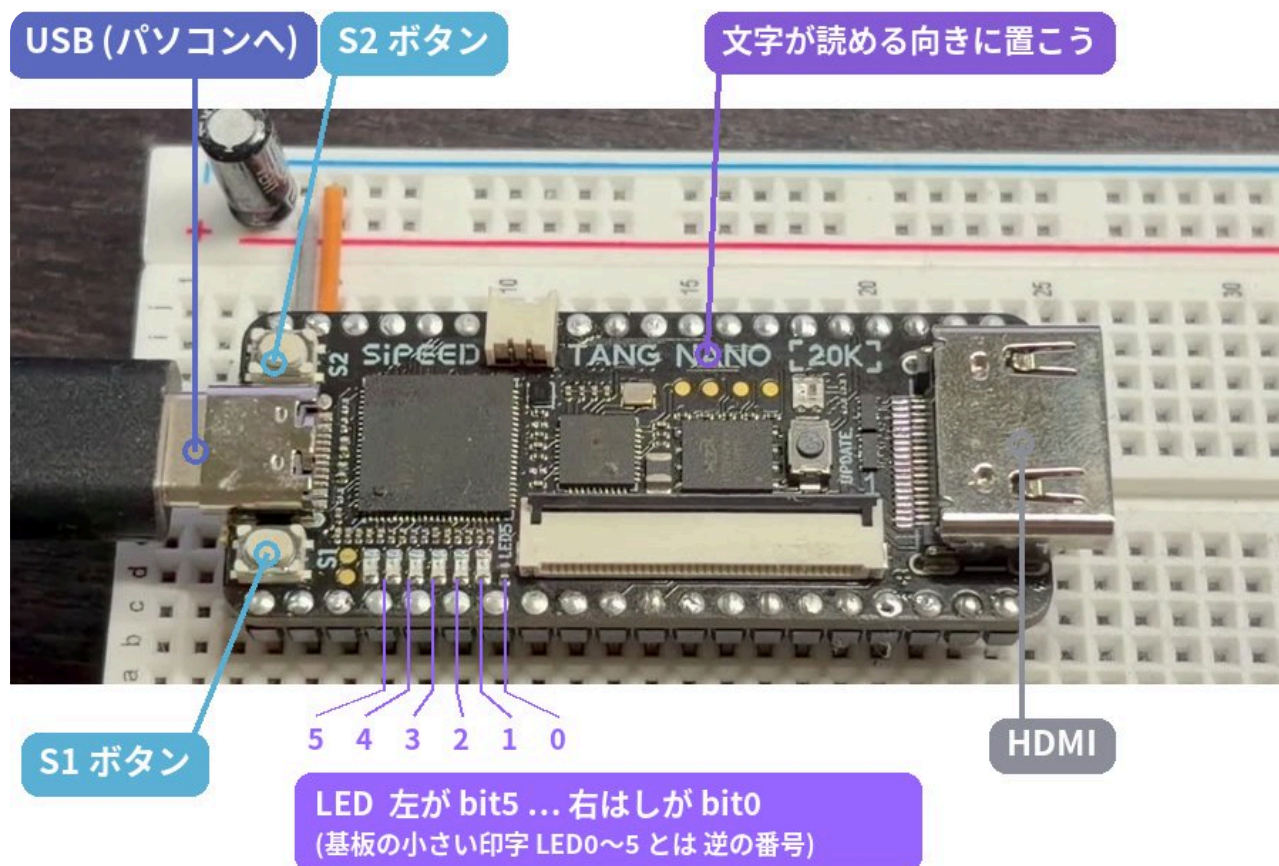
① このプロジェクトを ダウンロードする

1. GitHub の このプロジェクトの ページ (<https://github.com/tjm8874/TTA-8>) を 開く
2. 緑の「Code」ボタン → 「Download ZIP」
3. ZIP を 開いて (展開して)、好きな 場所におく

中身は こうなっています。

```
TTA-8-main/      ← ZIP を 開くと この 名前 (TTA-8 に 変えても OK)
├─ FPGA_Tool/   ← ボードに 書きこむ 道具 (ダブルクリックで 使う)
│   └─ Windows/
│       └─ Mac/
├─ debugger/    ← TTA-8 ラボ (index.html を ブラウザで 開く)
├─ hdl/         ← CPU の 設計図 (Verilog) と 書きこむ ファイル
├─ programs/    ← サンプルプログラム
└─ docs/        ← この せつめい書
```

② ボードの 向きを おぼえよう



「TANG NANO 20K」の文字が読める向き（USB が左）で置きます。この向きで、LED は右はしが bit0、左はしが bit5 です。プログラムで OUT 1 とすると、右はしの LED が光ります。

基板には小さく LED0~LED5 と印字されていますが、このプロジェクトでは 2進数と 合わせるために 逆の番号（右はしが 0）を使います。

③ Windows の じゅんび（はじめに 1回だけ）

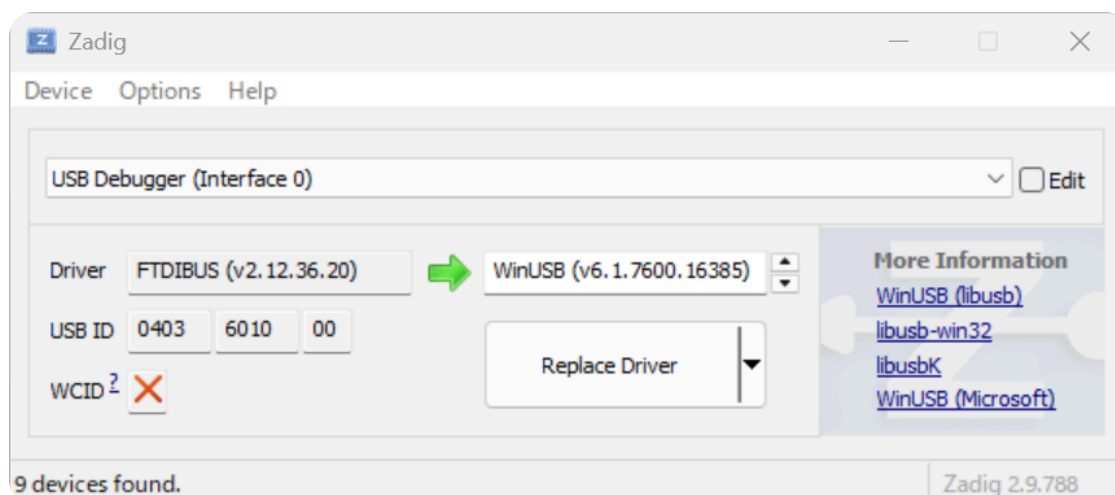
FPGA_Tool\Windows を開きます。

0_setup_driver.bat … USB ドライバの 設定

ボードに書きこむ道具（openFPGALoader）がボードと話せるように、Zadig（ザディグ）という道具でドライバを入れかえます。

1. ボードを USB でつなぐ
2. 0_setup_driver.bat をダブルクリック → 何かキーをおすと Zadig が開く
3. 「このアプリが変更を加えることを許可しますか」 → はい
4. メニューの Options → List All Devices にチェック

5. リストから **USB Debugger (Interface 0)** を えらぶ
6. 下の **USB ID が 0403 6010 00** になっていれば 正解
7. **Interface 1 は えらばない**
8. 右がわが **WinUSB** になっているのを 確認して **Replace Driver**



1. 「successfully」と 出たら 完了

リストに 出ないときは：「不明な USB デバイス」しか 出ないときは、ケーブルが 充電専用 かもしれません。べつの ケーブルに かえるか、パソコン本体の USB に 直接 さして みてください。

1_check_board.bat … ボードが 見えるか 確認

ダブルクリックして、model GW2A(R)-18(C) と 出れば OK です。

```
=====
ボードが つながっているか しらべます
=====

--- USB に つながっている もの ---
Bus device vid:pid      probe_type manufacturer serial      product
001 046      0x0403:0x6010 FTDI2232  SIPEED      2025030317 USB Debugger

--- FPGA を さがす ---
Jtag frequency : requested 6.00MHz    -> real 6.00MHz
index 0:
  idcode 0x81b
  manufacturer Gowin
  family GW2A
  model GW2A(R)-18(C)
  irlength 8

*** 「 GW2AR-18」 の よう な 名前 が 出 て い れ ば OK ! ***

Press any key to continue . . .
```

④ Mac の じゅんび（はじめに 1 回だけ）

Mac ではドライバの設定はいりません。「ターミナル」を開いて、FPGA_Tool/Mac の中でつぎを実行します。

```
bash 0_setup.sh          # openFPGALoader を 入れる（Homebrew が 必要）
bash 1_check_board.sh    # ボードが 見えるか 確認
```

ファイルをターミナルの画面にドラッグ&ドロップすると、場所を打たなくて済みます。
Homebrew がないときは、9章のやりかたで AI に聞いてみよう。

4. 動かしてみよう

書きこみかたは 2しゅるい

ファイル (Windows / Mac)	どうなる？	いつ 使う？
2_write_sram.bat / .sh	おためし。電源を 切ると 消える	何度も ためす とき
3_write_flash.bat / .sh	ずっと。電源を 切っても のこる	完成した とき

さいしょは **おためし** で やってみましょう。

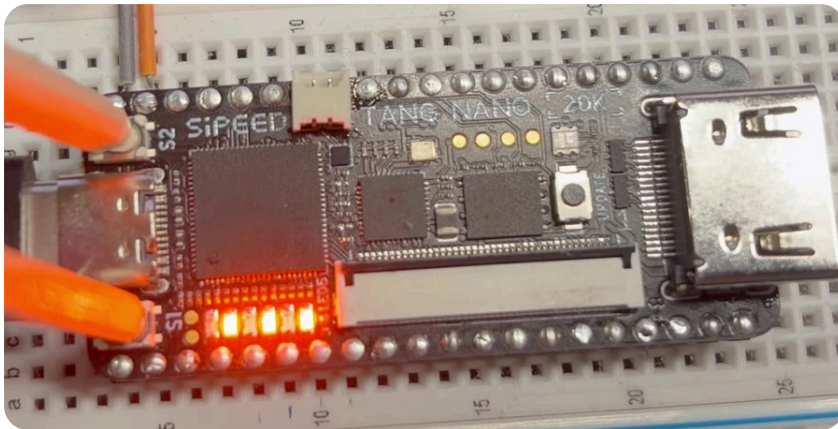
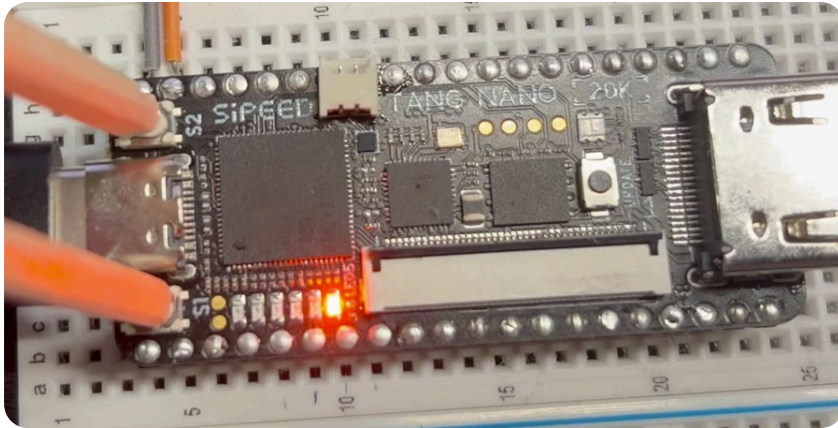
LED フラッシャーを 書きこもう

1. ボードを USB で つなぐ
2. FPGA_Tool\Windows\2_write_sram.bat を ダブルクリック (Mac は `bash 2_write_sram.sh`)
3. 「おわり！ LED を 見てみよう」と 出たら 成功

最初から 入っている `hdl/minimal_cpu.fs` には、LED フラッシャー の プログラムが 入っています。

ボタンを おしてみよう

おす ボタン	LED の 動き (0.5秒ごと)
なし	右はしが 点滅 <code>___○ ⇔ ___</code>
S1	光が 右から 左へ 流れる <code>___○ → ___○ → … → ○___</code>
S2	外から 内へ <code>○___ → _○_○ → __○○</code>
S1 と S2	交互に 点滅 <code>○_○_○ ⇔ _○_○_</code>



左：なにもおさないとき（右はしが点滅） 右：S1 と S2 を いっしょにおしたとき

電源を切っても動くようにする

`3_write_flash.bat` で書きこむと、USB をぬいてさしなおしても同じプログラムが動きます。少し時間がかかりますが、さいごに **Done** と **おわり！** が出れば OK です。

```
=====
Tang Nano 20K に かきこみ [ずっと]
※電源を きっても 消え ません。少し 時間 が かかります
=====
ファイル : E:\AI\Claude\Minimal_CPU\FPGA_Tool\Windows\...\hdl\minimal_cpu.fs

write to flash
Jtag frequency : requested 6.00MHz  -> real 6.00MHz
Parse file Parse E:\AI\Claude\Minimal_CPU\FPGA_Tool\Windows\...\hdl\minimal_cpu.fs:
Done
DONE
after program flash: displayReadReg 00006020
Memory Erase
Done Final
Security Final
Erase SRAM DONE
Jtag probe limited to 6000000 Hz
Jtag frequency : requested 10.00MHz  -> real 6.00MHz
Detail:
Jedec ID      : 0b
memory type   : 40
memory capacity : 17
Detail:
Jedec ID      : 0b
memory type   : 40
memory capacity : 17
RDSR : 0x00
WIP : 0
WEL : 0
BP : 0
TB : 0
SRWD : 0
flash chip unknown: use basic protection detection
start addr: 00000000, end_addr: 00090000
Erasing: [=====] 100.00%

Done
Writing: [=] 0.13%
Writing: [=====] 22.89%
Writing: [=====] 45.46%
Writing: [=====] 68.62%
Writing: [=====] 91.77%
Writing: [=====] 100.00%

Done

*** おわり ! LED を 見てみよう ***

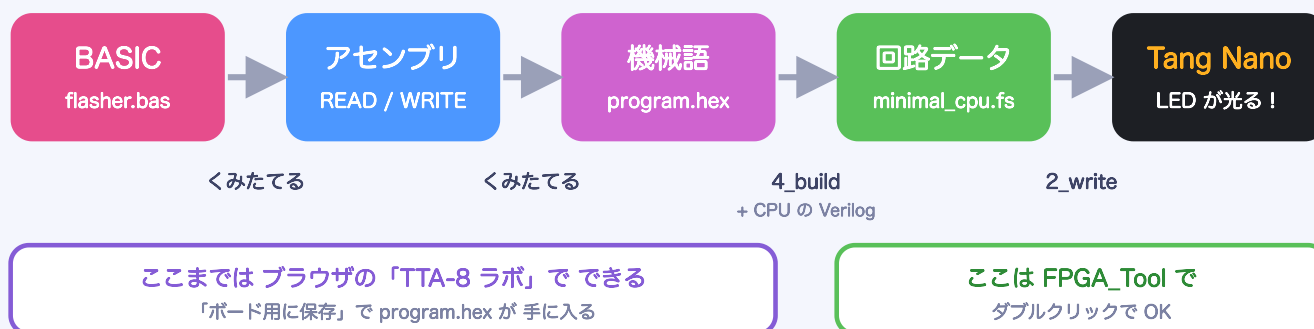
何か キー を おすと この 画面 を とじます。
```

黄色い文字で flash chip unknown と 出ますが、問題 ありません。

自分のプログラムを 書きこむには

ボードの中身は CPU (Verilog) + プログラム (program.hex) で できています。プログラムを 変えたら、CPU と いっしょに くみたて直す (ビルドする) ひつようがあります。

プログラムが ボードで 動くまで



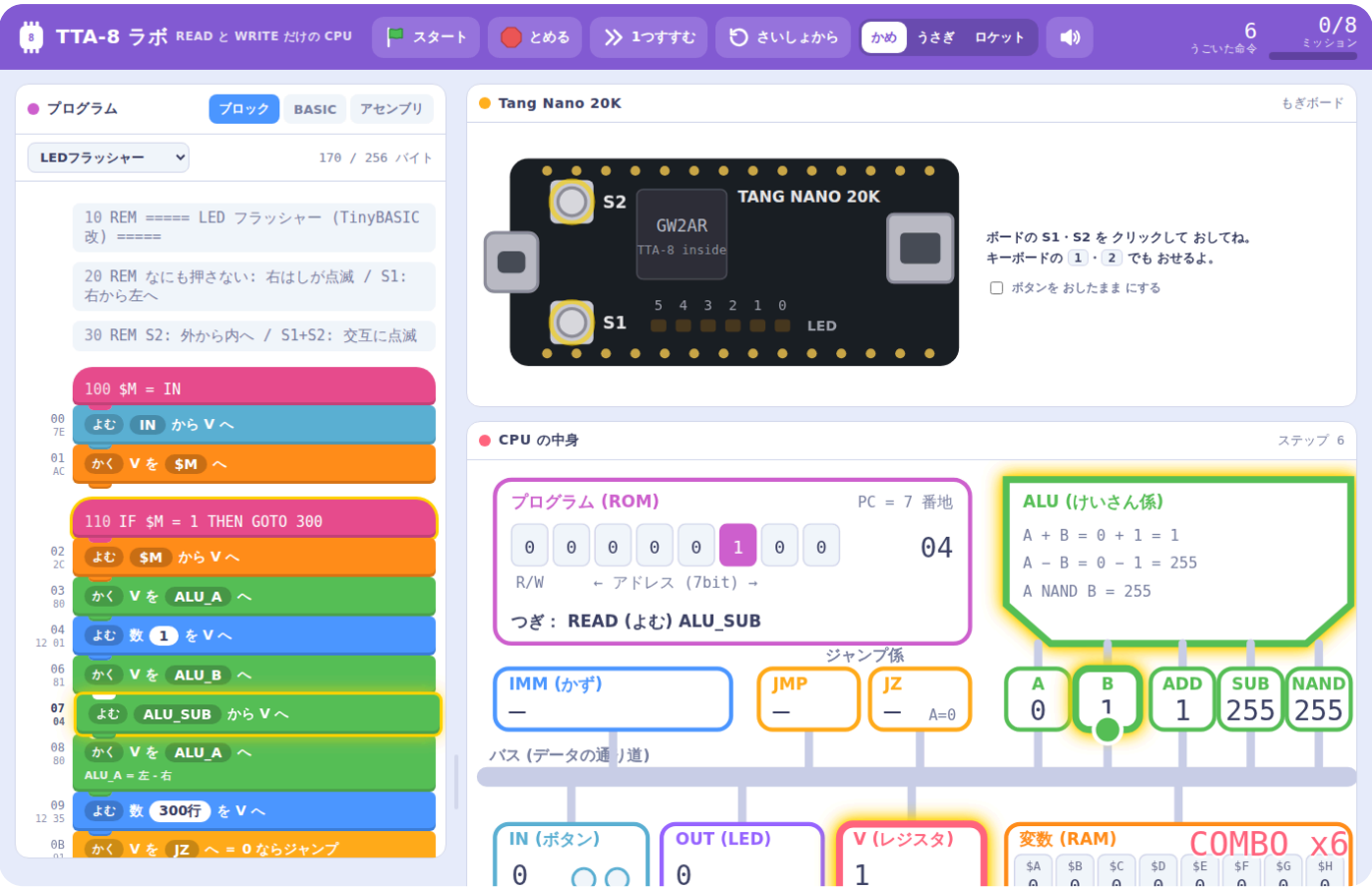
1. 5章の TTA-8 ラボでプログラムを作り、「ボード用に保存」をおす → program.hex が保存される

2. `program.hex` を `4_build.bat` の上にドラッグ&ドロップ (Mac は `bash 4_build.sh program.hex`)
3. 自動で `hdl/program.hex` にコピーされて、ビルドが始まります
4. **はじめてのときだけ**、ビルド道具 (OSS CAD Suite、約600MB) をダウンロードします
5. 「できた！」と出たら、`2_write_sram.bat` で書きこむ

5. TTA-8 ラボで 中を のぞこう

TTA-8 ラボ は、ブラウザで 動く TTA-8 の デバッガです。ボードがなくても、CPU の 中で 何が 起きて いるかを 1命令ずつ 見られます。

debugger/index.html を ダブルクリックすると、ブラウザで 開きます（インターネットはいりませ ん）。



画面の 見かた

場所	見えるもの
左：プログラム	ブロック・BASIC・アセンブリの 3つの 見かたで プログラムを 表示
右上：もぎボード	Tang Nano 20K の まね。S1・S2 を クリックでおせる
右中：CPU の 中身	PC・V・ALU・変数・IN・OUT が いまの 値と いっしょに見える
右下：ミッション	クリアすると ほしが もらえる

動かしかた

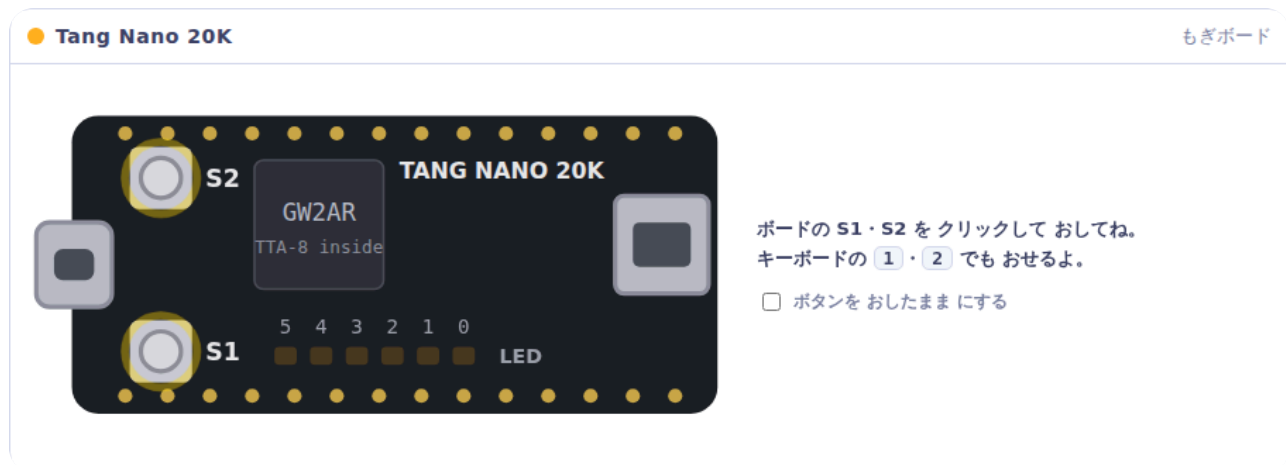
ボタン	キー	すること
スタート（みどりの旗）	スペース	動かす
とめる		止める
1つすすむ	→	1命令だけ 進める
さいしょから		リセット
かめ / うさぎ / ロケット		速さ。ロケット は 本物の ボードと 同じ 1秒に 1000命令

1つすすむ を おすと、V が 部品から 部品へ 数を 運ぶ ようすが、バス（データの 通り道）を 走る 玉で見えます。

- 玉が 部品 → V に 走ったら READ（よむ）
- 玉が V → 部品 に 走ったら WRITE（かく）

キャラクターが「いま 何を したか」を ことばで 教えてくれます。

もぎボード



本物の ボードと 同じ 向き・同じ 位置に、S1・S2・LED があります。キーボードの 1・2 でも ボタンをおせます。「ボタンをおしたままにする」に チェックすると、両方をおしたままに できます。

ブレークポイント

プログラムの 行の 左はしを クリックすると、そこで 止まる しるし（ブレークポイント）が つきます。ロケットで 走らせて、気になる ところで 止めてみましょう。

ミッション

● ミッション		0 / 8	
★ みどりの旗でスタート！	★ 「1つすすむ」を 10回 おそう 7/10		
★ JZ のジャンプが決まるのを見よう	★ S1 をおして 光を右から左へ流そう		
★ S2 をおして 外から内へ光らせよう	★ S1 と S2 を いっしょにおそう		
★ ブレークポイントで 止めてみよう	★ プログラムを書きかえて くみたてよう	COMBO x7	

「1つすすむを 10回 おそう」「JZ のジャンプが決まるのを見よう」など、8つのミッションがあります。ぜんぶクリアできるかな？

プログラムを 書きかえる

- BASIC タブ → かきかえる → 書きかえて くみたてる
- アセンブリ タブ → 書きかえて くみたてる
- できたら ボード用に保存 で `program.hex` を 保存 → 4章 の やりかたで ボードへ

6. TinyBASIC改 で プログラムを 書こう

TinyBASIC改 は、TTA-8 のために 作った 小さな BASIC です。パソコン（ブラウザ）の 上で、TTA-8 の 命令（READ / WRITE）に 翻訳されます。

はじめての プログラム

```
10 REM ===== はじめての たし算 =====  
20 $A = 3  
30 $B = 5  
40 $C = $A + $B  
50 OUT $C  
60 END
```

- 1行ずつ **行番号**（10, 20, 30 …）から 書きます
- **\$A** は **変数**（数を 1つ しまっておく 箱）
- **OUT** で LED に 出します。8 は **001000** と 光ります
- **REM** の うしろは メモ（プログラムには 関係 ありません）

変数と 数

もの	書きかた	せつめい
変数	\$A ~ \$Z 、 \$0 ~ \$5	ぜんぶで 32こ。0~255 の 数が 1つ 入る
数	12 、 0b101010 、 0x2A	10進・2進・16進。0~255

255 を こえると 0 に もどり、0 より 小さく になると 255 の ほうへ まわります（8bit だから）。

計算

書きかた	意味
<code>\$A + 1</code>	たし算
<code>\$A - 1</code>	ひき算
<code>\$A AND 0b000111</code>	どちらも 1 のビットだけ 1
<code>\$A OR 0b100000</code>	どちらかが 1 のビットは 1
<code>\$A NAND \$B</code>	AND の反対
<code>IN</code>	ボタン (S1 = 1、S2 = 2、両方 = 3)

左から 順に 計算します (かっこは 使えません)。

命令の一覧

命令	例	意味
代入	<code>\$A = \$B + 1</code>	変数に 入れる
<code>OUT</code>	<code>OUT 0b000001</code>	LED に 出す (右はしが bit0)
<code>IF ... THEN</code>	<code>IF \$M = 1 THEN GOTO 300</code>	同じ なら THEN の あとを やる
	<code>IF IN <> \$M THEN GOTO 100</code>	ちがう なら THEN の あとを やる
<code>GOTO</code>	<code>GOTO 100</code>	その 行へ とぶ
<code>GOSUB / RETURN</code>	<code>GOSUB 900</code>	その 行へ 行って、RETURN でもどってくる
<code>WAIT</code>	<code>WAIT 50</code>	50 × 0.01秒 = 0.5秒 まつ
<code>POKE / PEEK</code>	<code>POKE 127, 5</code>	アドレスに 直接 書く / 読む
<code>END</code>	<code>END</code>	おわり
<code>REM</code>	<code>REM メモ</code>	メモ

： で 区切ると、1行に いくつも 書けます (例： `$L = 1 : GOSUB 900`)。

ためしてみよう

LED で 2進数カウンター

```
10 REM ===== LED で 2進数カウンター =====
20 $C = 0
30 OUT $C
40 WAIT 50
50 $C = $C + 1
60 GOTO 30
```

0.5秒ごとに 1ずつ ふえて、LED が 2進数で 数えます。63 の つぎは？ 考えてから ためしてみよう。

かけ算 6 × 7

TTA-8 には かけ算の 部品がありません。でも **たし算を くりかえせば** できます。

```
10 REM ===== かけ算 6 x 7 (たし算のくりかえし) =====
20 $A = 6 : $B = 7 : $C = 0
30 IF $B = 0 THEN GOTO 70
40 $C = $C + $A
50 $B = $B - 1
60 GOTO 30
70 OUT $C
80 END
```

答えの 42 は **101010**。LED が 1つおきに 光ります。

LED フラッシャー (ボードに入っている もの)

`programs/basic/flasher.bas` を TTA-8 ラボの サンプルから えらんで、中を 読んでみましょう。

`IN` で ボタンを 読んで、`IF` で 模様を えらんでいます。

できないこと と その かわり

できないこと	かわりの 方法
<code><</code> <code>></code> で 比べる	<code><></code> (ちがう) を 使っ て くりかえす
かっこ ()	式を 2行に 分ける
<code>GOSUB</code> の 中 で <code>GOSUB</code>	もどき先を 1つしか おぼえられない
<code>PRINT</code> (文字を 出す)	画面がないので LED に <code>OUT</code> する

くわしい 文法は TinyBASIC改 文法書 を 見てね。

7. アセンブリで書いてみよう

BASIC は 便利ですが、CPU が ほんとうに やっているのは **READ** と **WRITE** だけです。その 命令を 直接 書く ことばを **アセンブリ** と いいます。TTA-8 ラボの **アセンブリ** タブで 書けます。

書きかた

```
; セミコロンの あとは メモ
LABEL:  READ  名前          ; 名前の 部品から V へ
        WRITE 名前          ; V から 名前の 部品へ
        READ  IMM, 数        ; 数を そのまま V へ (2バイト 命令)
        READ  IMM, LABEL    ; LABEL の 番地を V へ (ジャンプ先に 使う)
```

READ できる 名前	WRITE できる 名前
ALU_A ALU_B ALU_ADD ALU_NAND ALU_SUB IN IMM 変数	ALU_A ALU_B JMP JZ OUT 変 数

変数は \$A ~ \$Z、\$0 ~ \$5 です。

ジャンプのしかた

ジャンプは「行き先の 番地を V に 入れて、JMP に 書く」の 2命令です。

```
READ  IMM, LOOP    ; V = LOOP の 番地
WRITE JMP           ; PC = V → LOOP へ ジャンプ!
```

もし 0 なら ジャンプ は、先に ALU_A に 調べたい 数 を 入れておいて、JZ に 書きます。

```
READ  $N
WRITE ALU_A        ; ALU_A = $N
READ  IMM, DONE
WRITE JZ           ; ALU_A が 0 なら DONE へ。0 でなければ 次の 行へ
```

れんしゅう：5 から 0 まで カウントダウン

```
; 5 から 0 まで 数えて LED に出す
      READ IMM, 5
      WRITE $N          ; $N = 5
LOOP:  READ $N
      WRITE OUT          ; LED に $N を出す
      WRITE ALU_A        ; ALU_A = $N
      READ IMM, DONE
      WRITE JZ           ; ALU_A が 0 なら DONE へ
      READ IMM, 1
      WRITE ALU_B        ; ALU_B = 1
      READ ALU_SUB       ; V = $N - 1
      WRITE $N           ; $N = V
      READ IMM, LOOP
      WRITE JMP          ; LOOP へ もどる
DONE:  READ IMM, DONE
      WRITE JMP          ; ずっと 足踏み
```

TTA-8 ラボに 貼りつけて **くみたてる** → **かめ** の速さで スタート。LED が 5 (000101) → 4 → 3 → 2 → 1 → 0 と 変わって 止まれば 成功です。

ポイント：WRITE OUT のあと、V の中身は 変わらないので、そのまま WRITE ALU_A できます。V を じょうずに 使いまわすのが アセンブリの コツです。

チャレンジ

- 0 から 5 まで **数え上げる** ように 書きかえよう (ヒント：ALU_ADD と、5 を ひいて 0 か 調べる)
- BASIC で 書いた `$C = $A + $B` が どんな アセンブリ になるか、TTA-8 ラボの アセンブリ タブ で 見てみよう
- ALU_NAND だけで AND を 作れるかな? (ヒント：NAND を 2回)

8. CPU の設計図を 読もう

ここまで、READ と WRITE を使ってプログラムを動かしてきました。今度は、その命令を実行する回路を読みます。設計図は `hdl/tta8.v`。Verilog（ベリログ）という回路を書くことばで、コメントと空行をのぞくと 53 行です。

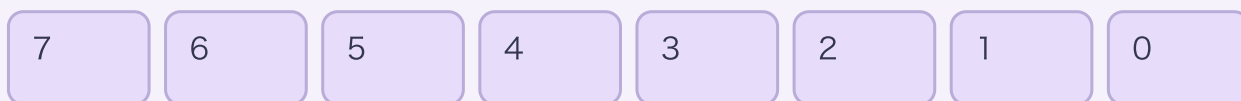
① 8bit の箱と、箱のならびを用意する

```
reg [7:0] v;  
reg [7:0] ram [0:31];
```

`[7:0]` は「7番から 0番までの 8マス」。`v` は数を運ぶための箱です。`ram [0:31]` は、その 8bit の箱を **32個** ならべる指定です。`ram[3]` と書けば 3番の箱を選べます。番号は 0 からなので、4個目です。

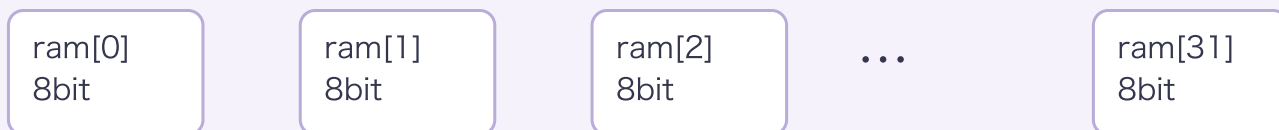
Verilog：ビットを選ぶ／箱をならべる

`reg [7:0] v;` → 8マスで一つの数



`v[7]` は左はしの 1bit。`v[6:0]` は残りの 7bit。

`reg [7:0] ram [0:31];` → 8bitの箱が32個



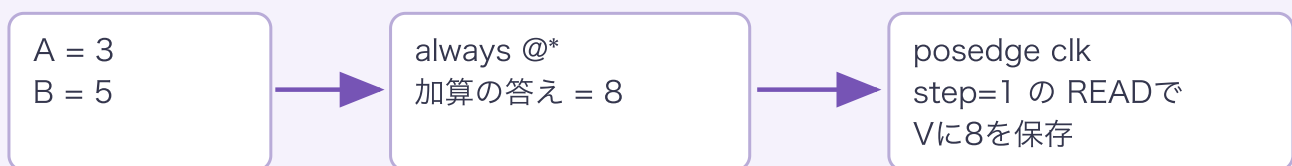
`[7:0]` は各箱の幅。`[0:31]` は箱の番号。

書き方	このCPUでの役目
<code>reg [7:0] v;</code>	クロックの合図で 値を覚える V
<code>reg [7:0] pc;</code>	実行する命令の バイト番地 を覚える PC
<code>reg [7:0] alu_a, alu_b;</code>	計算に使う A と B を覚える
<code>reg [7:0] rom [0:255];</code>	プログラム用の 256バイトの配列
<code>wire [7:0] inst = rom[pc];</code>	PC が選んだ命令を <code>inst</code> という名前で 取り出す

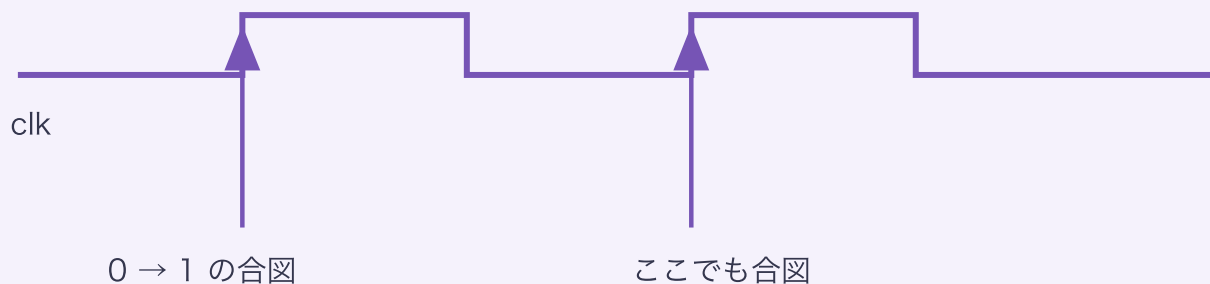
V は「数を運ぶ専用」のレジスタです。PC や ALU_A ・ ALU_B も 値を覚える部品です。なお、`reg` と書くだけで 必ず記憶回路になるわけではありません。どの `always` の中で、**どう値を決めるか** も関係します。次の図で 区別しましょう。

② 「いつも計算」と「合図で保存」を 分ける

計算はつながって動く。記憶は合図で変わる。



合図の前：答えが8でも、Vはまだ前の値。



```

always @* begin
    rdata = alu_a + alu_b;
end
  
```

これは 説明用に 足し算だけを 抜き出した例です。 `always @*` は、使っている値が変わると 答えも変わる回路を表します。A が 3、B が 5 なら、少しの回路の遅れのあとで 答えは 8 になります。実際の CPU では `case` を使い、足し算・引き算・ほかの部品の どれを読むかも 選びます。

```
always @(posedge clk) begin
    if (step) v <= rdata;
end
```

こちらは、クロックが **0 から 1 に変わる瞬間** に値を保存する回路です。posedge はこの立ち上がりを指します。さらに step が 1 のときだけ保存します。実際のCPUでは READ のときに V へ保存し、WRITE のときは書き込み先へ保存します。

= はこの例では「計算中の値を決める」、<= は「合図の直前の値を使い、保存する値を予約する」書き方です。この <= は「以下」という比較ではありません。

たとえば、同じクロック処理の中に 次の2行があったとします。

```
v <= rdata;
alu_a <= v;
```

合図の直前が v = 3、rdata = 8 なら、更新後は **V が 8、A が 3** になります。上の行で予約した 8 を、下の行がすぐ使うわけではありません。TTA-8 では READ と WRITE を別々の命令にしているので、この2つの保存を 1 命令では行いません。

③ 命令のビットを 分けて、読む相手を選ぶ

```
wire [7:0] inst = rom[pc];
wire      write = inst[7];
wire [6:0] addr = inst[6:0];
```

inst[7] は左はしの1bit。inst[6:0] は残りの7bitです。write が 0 なら READ、1 なら WRITE。addr が相手の番号です。

```
case (addr)
    7'h00: rdata = alu_a;
    7'h01: rdata = alu_b;
    7'h02: rdata = alu_a + alu_b;
    7'h03: rdata = ~(alu_a & alu_b);
    7'h04: rdata = alu_a - alu_b;
    7'h12: rdata = rom[pc + 8'd1];
    7'h7E: rdata = {6'b0, btn};
    default: rdata = is_ram ? ram[addr[4:0]] : 8'h00;
endcase
```

case は 番号ごとに 選ぶものを分けます。default は どの番号にも当たらないときです。最後の行の 条件 ? はい : いいえ は、RAM の番号なら RAM を読み、それ以外なら 0 を選びます。

記号	読み方
7'h12	7bitで表した 16進数の12（10進数では18）
8'd1	8bitで表した 10進数の1
& / ~	ビットごとの AND / 0と1の反転
{6'b0, btn}	6個の0と、2bitのボタンの値をつないで 8bitにする
== / !	等しいかを調べる / 条件を反対にする

ALU の答えは、A と B が変われば変わります。READ は、その時点で用意されている答えを V に覚えます。

④ クロックごとの変化を 追ってみよう

2章 の足し算を、0番地から入れた場合です。READ IMM は 命令と数で2バイト、ほかは1バイトです。

合図	実行前のPC	実行する命令	実行後のV	実行後のA / B	次のPC
1	0	READ IMM, 3	3	0 / 0	2
2	2	WRITE ALU_A	3	3 / 0	3
3	3	READ IMM, 5	5	3 / 0	5
4	5	WRITE ALU_B	5	3 / 5	6
5	6	READ ALU_ADD	8	3 / 5	7
6	7	WRITE OUT	8	3 / 5	8

4回目の合図で B が 5 になると、加算回路の答えは 8 になります。5回目の合図で その8を V に保存し、6回目で LED に渡します。WRITE のあとも V の値は残っています。

予想してみよう：3回目のあとで止めたとき、V と A は同じ値でしょうか？ 表をかくして考え、ラボに 2章 の命令を入れて確かめよう。

⑤ PCを進める予約と、ジャンプの予約

```
pc <= pc + 8'd1;  
// WRITE JMP の場合は、このあとで  
pc <= v;
```

同じ `always` の中で、同じ合図に対して 同じレジスタへ複数の `<=` を実行すると、**あとに実行した代入が採用されます**。だから JMP のときは `pc + 1` の代わりに `V` が次のPCになります。別々の `always` から同じレジスタへ書く、という意味ではありません。

JZ では `if (alu_a == 0)` が成り立ったときだけ `pc <= v` を実行します。A が 0 でなければ 最初の `pc + 1` が残り、次へ進みます。READ IMM の場合は `pc + 2` に予約し直して、数が入ったバイトを飛ばします。

⑥ リセットで、32個の変数も 0 にする

```
integer i;  
// always @(posedge clk) の中  
if (reset) begin  
    pc <= 0; v <= 0; alu_a <= 0; alu_b <= 0; led <= 0;  
    for (i = 0; i < 32; i = i + 1) ram[i] <= 0;  
end
```

`for` は 0番から31番まで、各箱を指定するための繰り返しです。この回路では **1回のクロックの合図で、32個すべてを0にします**。CPU が32命令かけて消すプログラムではありません。

実際のコードは `if (reset) ... else if (step) ...` なので、リセットを優先します。`step` が 0 でも、リセット中のクロックで初期化されます。起動時も再リセット時も変数は0になり、ラボの「さいしょから」と揃います。

⑦ ボードと つなぐ

`hdl/top_tangnano20k.v` は、ボードとCPUをつなぐファイルです。

- 電源を入れた直後は リセットする。
- 27MHz のクロックを数えて、1秒に1000回だけ `step` を1にする。
- ボタンの信号を クロックに合わせて受け取る。
- LED は 0で光るので、出力を `~led` で反転する。

ピンの対応は `hdl/tangnano20k.cst` に書いてあります。CPU は27MHzのクロックを受けますが、命令を実行するのは `step` が1の合図だけです。

「この回路だけでわり算もできるの？」は 発展1、「箱や番地を増やしたい！」は 発展2 で試して考えます。

9. AI に こうやって たのもう

こまったときは、AI（Claude や ChatGPT、Gemini など）に 聞いてみましょう。 **じょうずに 聞く コツ** は 3つです。

1. **何を しようとしているか** を 書く
2. **何が 起きたか**（エラーの 文字や 画面）を そのまま 見せる
3. **自分の パソコン**（Windows か Mac か）を 書く

そのまま 使える たのみかた

じゅんびで こまったとき

Tang Nano 20K という FPGA ボードに、openFPGALoader で 書きこもうとしています。
パソコンは Windows 11 です。
1_check_board.bat を 動かしたら、下の 文字が 出ました。どうすれば いいですか？
（ここに 黒い画面の 文字を 貼る）

Mac に Homebrew を 入りたいです。小学生にも わかるように、
ターミナルで 何を 打てば いいか 1つずつ 教えてください。

プログラムを 作りたいとき

AI は TinyBASIC改 を 知りません。 **文法書を いっしょに わたす** のが コツです。

下の 文法の「TinyBASIC改」で、LED 6こ を 使った プログラムを 書いてください。
S1 を おしたら LED が 左から 右へ 流れるように したいです。
LED は 右はしが bit0 です。1命令 0.001秒 の CPU です。
（ここに docs/tinybasic.md の 中身を 貼る）

この TinyBASIC改 の プログラムで、LED が 思った とおりに 光りません。
やりたいこと：（ここに 書く）
プログラム：（ここに 貼る）
TTA-8 ラボで 動かすと、（ここに 起きたことを 書く）

しくみを 知りたいとき

TTA-8 という、命令が READ と WRITE だけの CPU を 勉強しています。
「JZ に 書くと ALU_A が 0 の ときだけ ジャンプする」の 意味を、
小学5年生に わかるように たとえ話で 説明してください。

hdl/tta8.v の この 部分が わかりません。1行ずつ 説明してください。
(ここに Verilog を 貼る)

改造したいとき

TTA-8 の Verilog (下に 貼ります) に、ブザーを 鳴らす 出力を 1つ 足したいです。
アドレス 0x7D に 書くと、ブザーの ON/OFF が 変わるように したいです。
Tang Nano 20K の どの ピンに つなげば いいかも 教えてください。
(ここに hdl/tta8.v と hdl/top_tangnano20k.v を 貼る)

気をつけること

- AI の 答えが **まちがっている** こともあります。TTA-8 ラボで ためして、動くか 確かめよう
- **名前・住所・パスワード** などは 書かないこと
- わからないところは「もっと かんたんに」「たとえ話で」と たのみなおして OK

10. つぎの ぼうけんへ

ここまで 読んだ キミは、

- CPU が **命令を 1つずつ こなす** しくみ
- **READ と WRITE** だけでも 計算・判断・くりかえしができる こと
- その CPU を **FPGA の 中に 作って 動かす** 方法

を知っています。これは 大人の エンジニアでも しっかり 説明できる 人が 少ない ことです。

ブレッドボードで 回路を ふやそう

Tang Nano 20K の まわりには、たくさんの **ピン（足）** が ならんでいます。ブレッドボードに させば、ボードの **外がわに いろいろな 回路を つなげられます**。

つなぐもの	できること	TTA-8 の 改造アイデア
LED を もっと	8こ、16こ の イルミネーション	OUT を 8bit に ふやす
ボタン・スイッチ	入力を ふやす	IN に ビットを 足す
ブザー	音を 鳴らす	新しい アドレスに「音」の 部品を 作る
7セグメント LED	数字を 表示する	変数の 中身を 数字で 見る
モーター・サーボ	ものを 動かす	ロボットの 頭脳に する
温度・光センサー	まわりの ようすを 読む	センサーの 値で LED の 色を 変える

TTA-8 に 部品を 足すのは かんたんです。 **新しい アドレスを 決めて、READ か WRITE で つなぐだけ**。それが TTA（運ぶと 動く しくみ）の いいところです。

回路を つなぐ ときは、**USB を めいてから**。Tang Nano 20K の ピンは **3.3V** です。5V の 部品を つなぐ ときは 大人と いっしょに 調べよう。

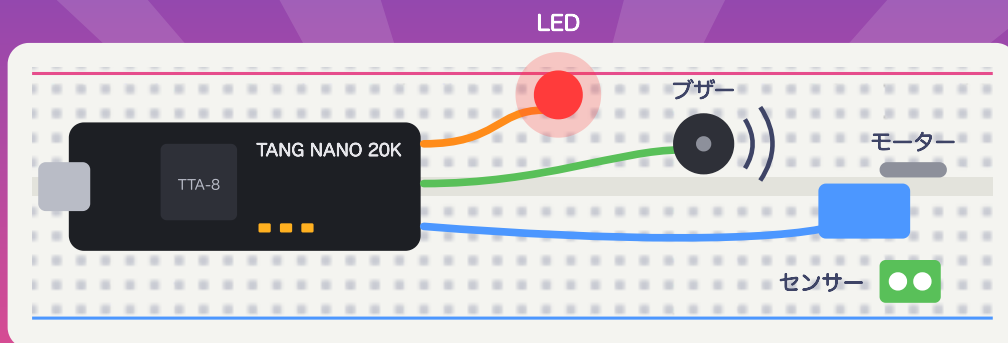
チャレンジ リスト

- □ LED フラッシャーに 新しい 模様を 足す
- □ 2進数カウンターを 0.1秒ごとに する
- □ かけ算の 答えが 63 を こえたら どうなるか 予想して ためす
- □ ALU に「かけ算」の 部品を 足して、アドレス 0x05 で 読めるように する
- □ ブザーを つないで、ボタンで ドレミを 鳴らす

- □ 自分で考えた 命令セットの CPU を 作る

おれたちの ぼうけんは これからだ！

ブレッドボードで ボードの外に 回路を つなげよう



TTA-8 のつづきは キミがつくる

ブレッドボードがあれば、このボードの外がわに 色々な 回路を 付けられる。

おれたちの ぼうけん（電子工作）は これからだ！

発展1. ADD・SUB・NAND で どこまで計算できる？

ここは 本編を読んでからの寄り道です。今のTTA-8の回路のまま 試せます。足し算・引き算・NAND に、記憶と条件分岐を組み合わせると、専用の回路がない計算も プログラムで作れます。

繰り返しは、前の命令へ戻ることで作る

「残り回数」を変数に入れ、1を引き、0か調べて戻れば、決まった回数の繰り返しです。7章 のカウントダウンは、そのままこの仕組みを使っています。

残り回数	すること
3	仕事を1回して、残りを2にする
2	仕事を1回して、残りを1にする
1	仕事を1回して、残りを0にする
0	JZ でおわりへ進む

足し算を繰り返すと かけ算になります。では、わり算は？

17個を、5個ずつの組に分ける

17個のおはじきを、1個ずつ数えて組に入れます。5個たまるたびに「できた組」を1増やし、組の中の数を0に戻します。

数え終わった個数	できた組（商）	今の組に入っている数（余り）
5	1	0
10	2	0
15	3	0
17	3	2

だから $17 \div 5 = 3$ あまり 2 です。「大きい・小さい」を直接調べる命令がなくても、「5と同じになったか」で組を作れます。

```

10 REM 17 / 5 : $Q = しょう, $R = あまり, $E = 0でわるエラー
20 $N = 17 : $D = 5 : $Q = 0 : $R = 0 : $E = 0
30 IF $D = 0 THEN $E = 1 : GOTO 120
40 IF $N = 0 THEN GOTO 120
50 $N = $N - 1
60 $R = $R + 1
70 IF $R <> $D THEN GOTO 40
80 $R = 0
90 $Q = $Q + 1
100 GOTO 40
120 OUT $Q
130 END

```

ラボのサンプル「わり算 17÷5」を選べます。ファイルは `programs/basic/divide.bas` です。実行すると商の3がLEDに出ます。ラボの変数で `$Q = 3`、`$R = 2` を見つけましょう。余りをLEDで見なければ、120行を `OUT $R` に変えます。`$N` は残り個数なので、最後は0になります。

30行は0で割る場合を分ける処理です。そのときは計算せず、`$E` を1にします。このプログラムで扱う数は0～255、割る数は通常1～255です。

試そう：20行を変えて、20÷5、3÷5、0÷5、17÷0を試そう。商だけでなく、余りと `$E` も確認してください。

よく使う「割る数を、引けるだけ引く」方法も作れます。ただし8bitの引き算では 3-5 が254に回り込むため、引いた値が0でないだけでは「まだ引ける」と判断できません。ここでは比較の仕組みを追加せずに追えるよう、1個ずつ数える方法を選びました。

NAND から NOT・AND・OR を作れる

NAND は「AND の答えを反転する」演算です。8bitの各桁で 同じことを行います。まず1bitだけを見ます。

A	B	A AND B	A NAND B
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

作りたい演算	NAND での作り方
NOT A	A NAND A
A AND B	T = A NAND B として、T NAND T
A OR B	(A NAND A) NAND (B NAND B)

たとえば A=12 (00001100)、B=10 (00001010) なら、AND は8、OR は14です。表の中のかっこは説明用です。TinyBASIC改では かっこを使えないので、途中の値を変数に保存します。

```

10 $A = 12 : $B = 10
20 $C = $A NAND $B
30 $C = $C NAND $C
40 OUT $C
50 END

```

これは NAND だけで AND を作り、8を出す例です。途中の値を保持するレジスタやRAMも、計算を組み立てる大事な部品です。

「なんでも」は、手順と数の範囲を考えて使う

NAND は、ビットの論理演算を組み立てる基本になります。ADD・SUB に記憶・JZ・JMPを組み合わせると、かけ算やわり算のように 手順のある計算を作れます。 **ADD・SUB・NANDの3回路だけで、記憶や分岐なしに何でもできる、という意味ではありません。**

TTA-8では 数は8bit、変数は32個、プログラムは256バイトです。大きな数は何個かの箱に分けて扱えますが、そのための命令と記憶場所も必要になります。速さや容量が足りなくなったら、回路やメモリをどう増やすかを 次の発展編で考えましょう。

発展2. CPU の箱と番地を 大きくするには？

ここは **回路を改造するときの設計メモ** です。以下の新しいポートや仕組みは、現行のTTA-8とラボにはまだありません。そのまま入力して動くサンプルではなく、「どこを変える必要があるか」を考える補題です。

16bitにしたら、何が変わる？

8bitの数の箱を16bitにすると、0～65,535を一つの箱に入れられます。ただし **v** の宣言だけを変えても、ほかの8bitの箱へ渡した時点で上位の桁が落ちます。

大きくする部分	一緒に考えること
V、ALU_A、ALU_B、RAM の各要素	計算結果とデータの通り道も16bitに揃える
定数の読み込み	今の IMM は後続1バイト。大きな数を2バイトで読むなら、順序とPCの進み方を決める
LEDへの出力	今は下位6bitだけ。16bit化してもLEDが自動で増えるわけではない
PCとプログラムROM	データ幅とは別に決める。Vを16bitにしてもプログラム容量は自動では増えない
アセンブラ、BASIC、ラボ	数の範囲、命令の長さ、表示、計算結果の扱いを揃える

命令を8bitのままにして、運ぶデータだけ16bitにする設計もできます。IMMを後続2バイトに変える案なら、命令1バイト+定数2バイトで、次のPCは3進みます。既存のプログラムは新しい形式で作り直します。

外のメモリには「番地の箱」と「データの窓口」を作る

今のRAMは32個で、命令に書いた番号で直接選びます。外のメモリをたくさん読み書きするなら、先に**アクセスしたい番地をレジスタに覚えさせる**方法があります。ここでは、この番地を覚える箱をアドレスインデックスレジスタと呼びます。

番地を覚える箱で、メモリの場所を選ぶ



先に番地を設定 → 同じ窓口でデータを読み書き（改造案）

たとえば8bitデータのまま、番地用に8bitの箱を二つ作ります。

新しく作るポートの例	WRITE	READ
MEM_LO	番地の下位8bitを保存	下位8bitを読む
MEM_HI	番地の上位8bitを保存	上位8bitを読む
MEM_DATA	選んだ番地へデータを書く	選んだ番地のデータを読む

； 改造後の使い方の案（現行版では動きません）

```
READ IMM, 0x12
WRITE MEM_HI
READ IMM, 0x34
WRITE MEM_LO
READ MEM_DATA      ; メモリの 0x1234 番地を V へ
WRITE $A
```

16bitの番地なら 65,536か所を選べます。1か所1バイトなら64KiBです。命令の7bitアドレスは、この**窓口の番号**を選ぶために使います。窓口が少なくても、その先の場所は多く選べるわけです。

ただし外付けメモリには、信号を出す回路やピン、読み書きのタイミング制御も必要です。応答が間に合わないときはCPUを待たせます。その間に同じWRITEを何度も発行しない仕組みも必要です。番地を覚える箱だけを追加すれば、メモリがつながるわけではありません。

考えよう：READのあとに番地を1増やす機能があれば、連続したデータを読むプログラムはどう短くなるでしょう？

入出力も「選ぶ窓口」と「使う窓口」に分けられる

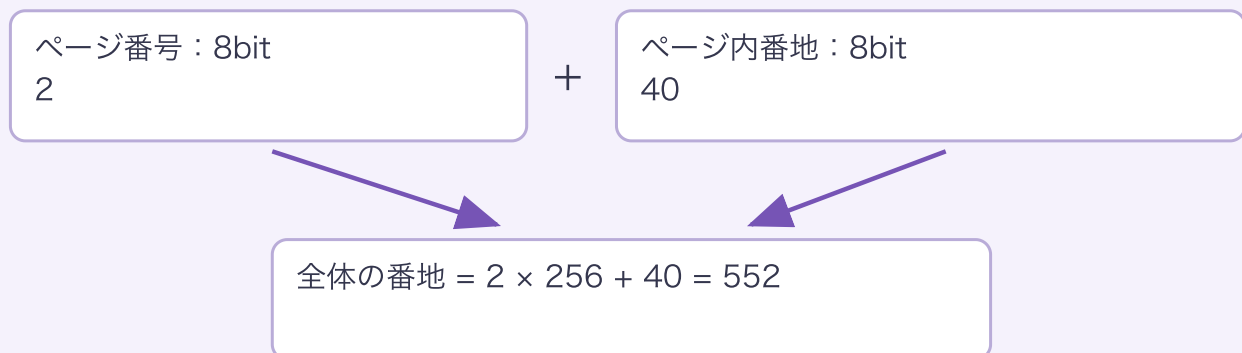
同じ考え方で `IO_INDEX` と `IO_DATA` を作れます。先に `IO_INDEX` へ装置の番号を書き、`IO_DATA` でその装置を読み書きします。番号を受け取る選択回路が、LED・ブザー・センサーなどへ振り分けま

す。
8bitの番号なら256か所、16bitなら65,536か所を選べます。命令に直接書ける128個のポート数を超えて拡張できるのが利点です。容量は番号の幅や接続先の回路で決まり、物理ピンや通信速度にも限りがあります。外部の通信回路を使って装置を増やす場合は、その通信手順も作ります。

プログラムは、ページ番号とページ内番地で選ぶ

今は8bitのPCだけなので、0～255番地です。256バイトを1ページとして、上位に8bitのページ番号を付ければ、256ページを選べます。全部で65,536バイト、64KiBのプログラム空間になります。

ページ番号とページ内番地で、命令の場所を選ぶ



順に進む：ページ2・255番地 → ページ3・0番地

ジャンプ：予約したページと番地を、同じ合図で切り替える

通常の実行では、ページ内番地255の次は、次ページの0番地へ進めます。`READ IMM` がページ末尾に来た場合も、次のページから定数を読み、全体のPCを2進める設計にします。ページ255の末尾を超えた場合にどうするかも決めます。

ジャンプ先の上位と下位を別々に書くなら、途中で命令を読むページだけが切り替わらないようにします。一つの案は「次のジャンプ先ページ」の箱を別に用意することです。

```
; 改造後の設計案。NEXT_PAGE は予約するだけ
READ IMM, 2
WRITE NEXT_PAGE
READ IMM, 40
WRITE JMP ; ここでページ2・番地40へ同時に切り替える
```

JZなら、条件が成立したときだけページと番地を同時に切り替えます。成立しなければ今のページの次の命令へ進みます。この案では毎回ジャンプ前にページを指定する約束にし、予約値の取り違えを防ぎます。ラベルをこの命令列に翻訳する処理も、アセンブラ側に必要です。

ページ番号を固定の窓へ切り替える「バンク切り替え」という別案もあります。どちらを選んでも、ROM本体・PCの更新・IMM・ジャンプ・ツールを一緒に設計します。

改造は、一つずつラボと回路を揃える

最初の改造には、10章の演算を一つ追加する課題が向いています。そのあとで、次のように範囲を広げられます。

1. FPGA内部の小さなRAMに インデックスでアクセスする。
2. 選ぶ番号を増やし、ラボと回路で同じ読み書きになるか確かめる。
3. 応答を待つ仕組みを作ってから、外付けメモリへ進む。
4. プログラムをページ化し、ページ境界のIMMと条件分岐を試す。

これらの拡張は「今のCPUを理解する」ための必須条件ではありません。今ある制限が、どの箱や配線の幅から来ているかを説明できたら、設計を変える入口に立っています。

A. こまったとき

こまったこと	ためすこと
Zadig に「不明な USB デバイス」しか 出ない	充電専用ケーブルかも。データ通信できる ケーブルに かえる / パソコン本体の USB に さす / USB-A ⇔ USB-C ケーブルを ためす
1_check_board で ボードが 見つからない	0_setup_driver を やったか 確認。Interface 0 を WinUSB に したか 確認
書きこめたのに LED が 動かない	もう一度 2_write_sram を ためす。USB を さしなおす
なにも おしてないのに LED が 交互に 点滅	ボタンの 向きの 設定が 逆。hdl/top_tangnano20k.v の BTN_ACTIVE_LOW を 変える (Tang Nano 20K は 0 で 正しい)
光る 向きが 左右 逆	ボードの 向きを 確認 (「TANG NANO 20K」が 読める 向き)
4_build の ダウンロードが 止まる	インターネット接続を 確認して もう一度。社内・学校の ネットワークでは 止められることも
TTA-8 ラボで「くみたて 失敗」	下の 赤い 文字を 読む。わからなければ 9章 の やりかたで AI に 聞く
Windows で 文字が 化ける	Windows 10 の 古い版では 起きることが あります。Windows Update を してみてください

B. 用語集

ことば	意味
CPU	命令を 1つずつ こなす 部品
FPGA	設計図を 書きこむと その回路に なる チップ
ビット (bit)	0 か 1 が 入る 1マス
バイト	8ビット。0~255
2進数 / 16進数	0と1だけの 数 / 0~9 と A~F の 数 (0x を つける)
レジスタ	CPU の 中の 小さな 箱。TTA-8 では V・PC・ALU_A・ALU_B など
PC (プログラムカウンタ)	実行する命令のバイト番地を おぼえる 箱
ALU	計算を する 係
アドレス	部品や メモリの 番号
ROM / RAM	プログラムを しまう ところ / 変数を しまう ところ
バス	データの 通り道
アセンブリ	CPU の 命令を そのまま 書く ことば
コンパイル	BASIC などを 機械語に 翻訳する こと
Verilog	回路を 書く ことば
ビルド	Verilog と プログラムから 書きこみ用 ファイルを 作る こと
クロック	レジスタに値を保存するタイミングの基準になる信号
インデックスレジスタ	アクセスする場所の番号を覚える箱 (発展2の改造案)
ページ	大きなプログラム空間を分けたまとまり (発展2では256バイト)

C. 大人の方へ（保護者・先生・開発者）

構成

パス	内容
hdl/tta8.v	CPU 本体（コメント・空行除き 53 行）
hdl/top_tangnano20k.v	ボード用トップ（27MHz → 1kHz ステップ、ボタン同期、LED 反転）
hdl/tangnano20k.cst	ピン割り当て（LED は bit0 = pin 20 … bit5 = pin 15 に反転）
hdl/program.hex	ROM 初期値（256 行の 16 進）
hdl/minimal_cpu.fs	ビルド済みビットストリーム（LED フラッシャー）
tools/	tta8asm.js（アセンブラ）、tbasic.js（TinyBASIC改 コンパイラ）、tta8sim.js（エミュレータ）、test.js
debugger/	TTA-8 ラボ（src/lab.html を build.py で index.html に 1 ファイル化）
FPGA_Tool/	書き込み・ビルド用スクリプト、openFPGALoader（Windows 版 同梱）、Zadig
sim/tb_flasher.v	iverilog 用テストベンチ
docs/architecture.md / docs/tinybasic.md	仕様書・文法書

コマンドラインでのビルド

OSS CAD Suite（yosys / nextpnr-himbaechel / apicula）を使います。FPGA_Tool/*/*_build と同じ内容です。

```
cd hdl
yosys -p "scratchpad -set abc9.xaiger 1; read_verilog top_tangnano20k.v tta8.v;
synth_gowin -top top_tangnano20k -json build/top.json"
nextpnr-himbaechel --freq 27 --json build/top.json --write build/pnr.json \
  --device GW2AR-LV18QN88C8/I7 --vopt family=GW2A-18C --vopt cst=tangnano20k.cst
gowin_pack -d GW2A-18C -o minimal_cpu.fs build/pnr.json
openFPGALoader -b tangnano20k minimal_cpu.fs          # SRAM
openFPGALoader -b tangnano20k -f minimal_cpu.fs        # Flash
```

`scratchpad -set abc9.xaiger 1` は、Windows 版 OSS CAD Suite (2026-09-29) で ABC9 が XAIGER2 の読み込み時にアサーションで落ちる問題の回避策です (旧 XAIGER 経路を使う)。

2026-09-30の再ビルド (同梱フラッシャー、RAM初期化あり) では、LUT4 1,269 / 20,736、DFF 342 / 15,552。27MHzのタイミング制約を満たしています。これは配置配線ツールの結果で、実機測定値ではありません。回路やプログラムで使用量は変わります。

プログラムのコンパイル (Node.js)

```
node tools/tbasic.js programs/basic/flasher.bas # → .asm / .hex / .lst
node tools/tta8asm.js programs/flasher.asm      # → .hex / .lst
node tools/test.js                               # コンパイラのテスト
node tools/test-lessons.js                       # わり算48ケース・教材例・初期化
sh tools/test-hdl.sh                             # Verilogの初期化・わり算
(iverilog/vvpが必要)
```

Gowin 公式ツールを使う場合

Gowin EDA (Education 版) でも `hdl/*.v` と `hdl/tangnano20k.cst` からビルドできます。ただし Zadig で WinUSB に置き換えた後は Gowin Programmer から認識しにくくなります。戻すときはデバイスマネージャーで該当デバイスのドライバを削除して再接続してください。

動作確認

2026-09-29、実機 (Tang Nano 20K) で SRAM・Flash 書き込みとも、なし / S1 / S2 / S1+S2 の全モードの動作を確認しています (初版)。RAM初期化を追加した版は、別途実機での確認が必要です。